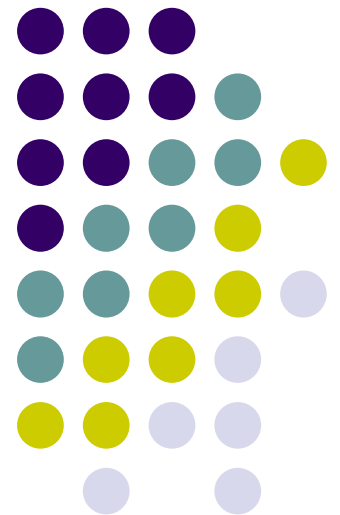
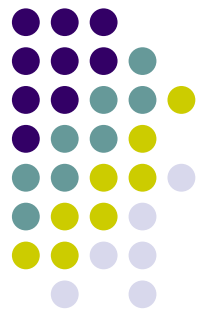


Scheduling a congested link

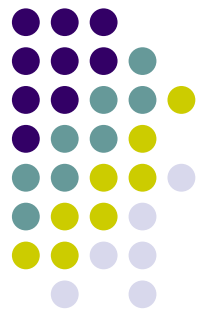




Scheduling

- Where?
 - Wherever contention for resources occurs.
 - We will concentrate on the output of a switch.
- Goal: Decide which is processed when.
 - Online: Decide without knowing what comes next
 - Offline: see complete input, then decide
- Desired Properties
 - Performance guarantees.
 - Offer different Quality of Service (QoS)
 - Fair resource sharing → **protection**

Priority Queuing



How:

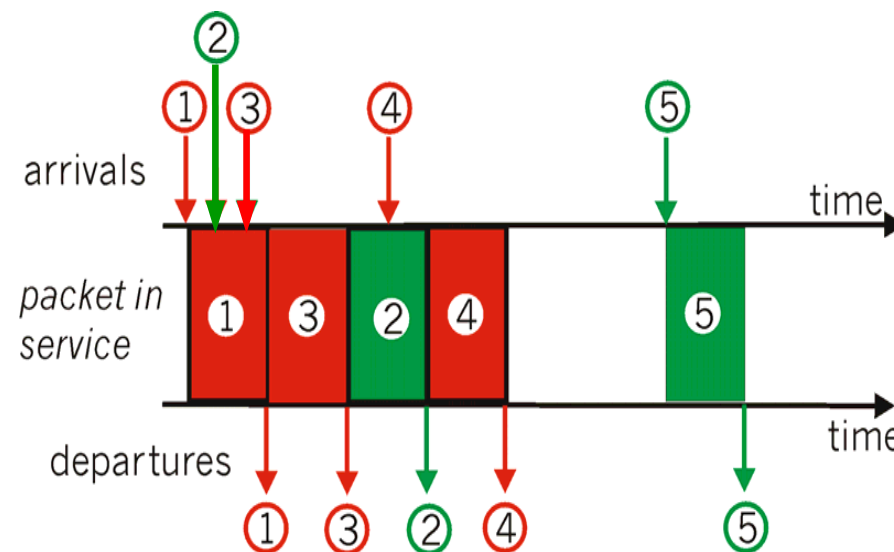
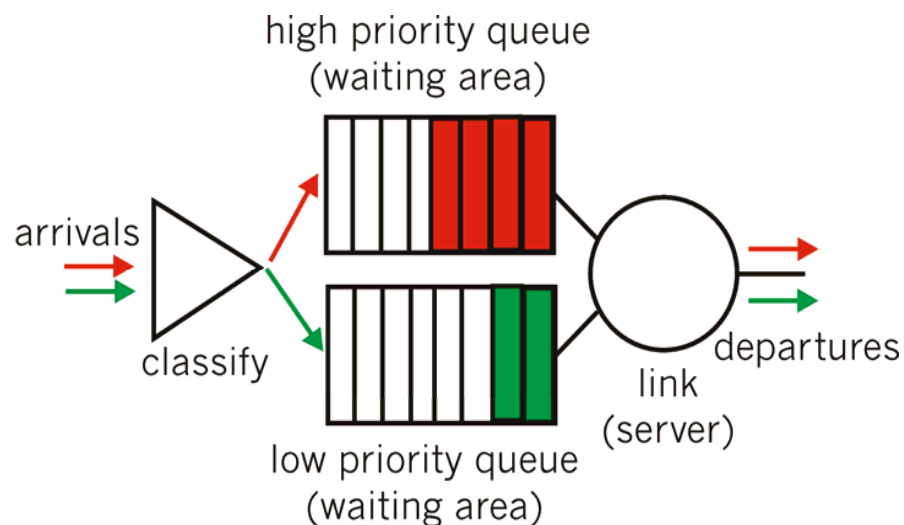
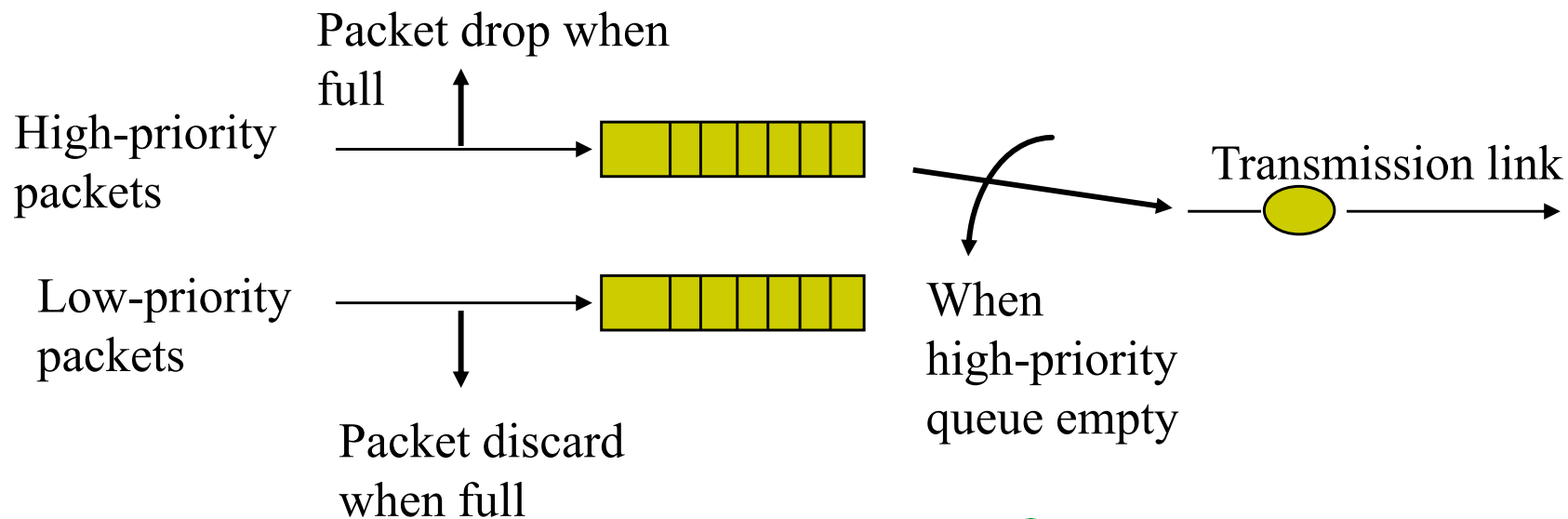
- A priority level is assigned to each packet upon arrival
- Packets transmitted in ascending order of priority levels.
 - level i serviced only if no packet of level $< i$.

Variants:

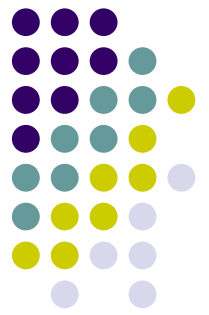
- **Preemptive and non-preemptive**



Priority Queuing (2 levels)



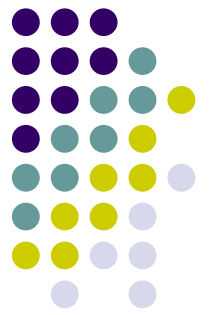
Fairness Issues: Discrimination or Equality?



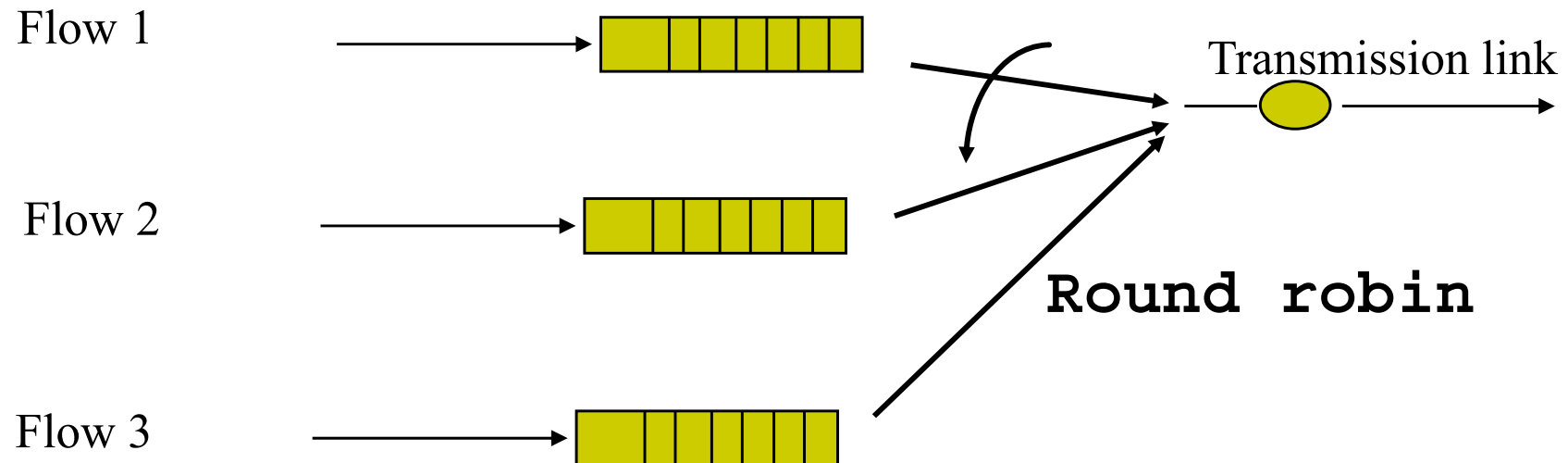
With strict priority:

- Highest priority has the
 - lowest delay,
 - highest throughput,
 - lowest loss
- Problems:
 - Lower priority classes may be **starved** by higher priority
 - How to assign priorities?
 - Also: how to schedule within a class?

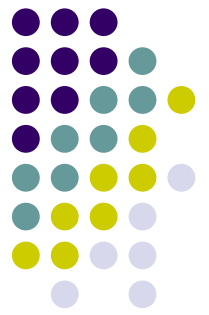
Complete Equality: Round Robin



Round Robin: serve one packet from the next non-empty class, in cyclic order

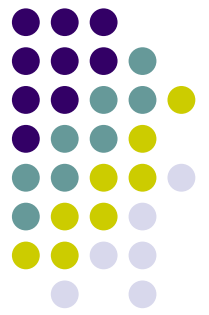


Useful hardware: Jump to next non-empty queue



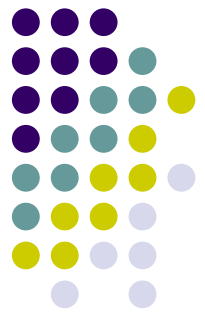
Round Robin

- Mode of work:
 - Classify incoming traffic into flows (source-destination pairs)
 - Round-robin among flows
- Advantage:
 - Protection against heavy users (full?)
- Issues:
 - Packets of different lengths
 - Too much equality...



Weighted Round-Robin

- Weighted round-robin
 - Each flow i gets weight $w_i \in \mathbb{N}$ (unweighted: $w_i = 1$)
 - Flow i can send w_i packets in a period.
 - Period length: $W = \sum_i w_i$
- Issues:
 - Variable packet size.
 - Fair only over time scales longer than W .
 - If the w_i 's are large, may get long periods of unfairness.

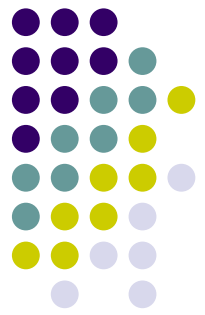


Deficit Round Robin

Choose a parameter “**quantum**”: # bits to serve from a connection.

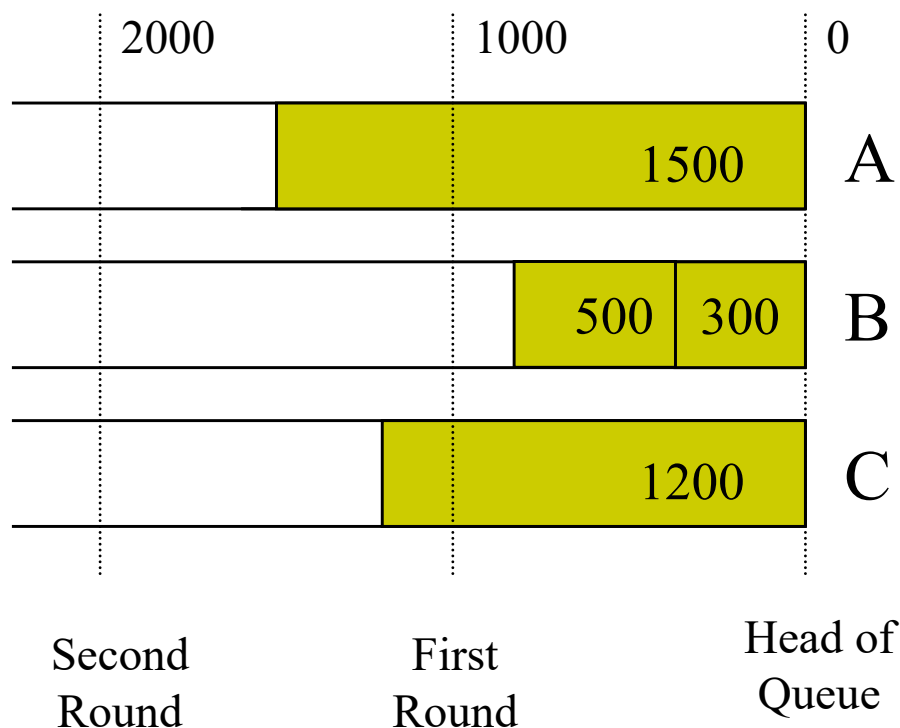
- For each flow f : variable credit_f , initially 0.
- Go over flows in turn. For each flow f :

- If there is no packet to send then
 $\text{credit}_f := 0$ // *to remain fair, no savings to later*
- Else
 $\text{credit}_f += \text{quantum};$
while ($s := \text{size of HOL packet}$) $\leq \text{credit}_f$
 send HOL packet;
 $\text{credit}_f -= s;$

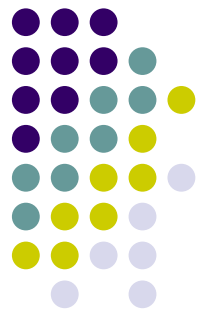


Deficit Round Robin

Queues status: in picture
Quantum size : 1000 byte
Credits: all 0

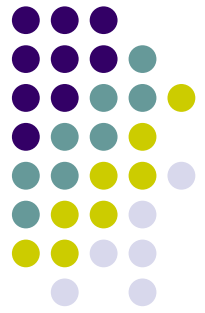


- 1st Round
 - A's credit : 1000
 - B's credit : 200, served twice
 - C's credit : 1000
- 2nd Round
 - A's credit : 500, served
 - B's credit : 0
 - C's credit : 800, served

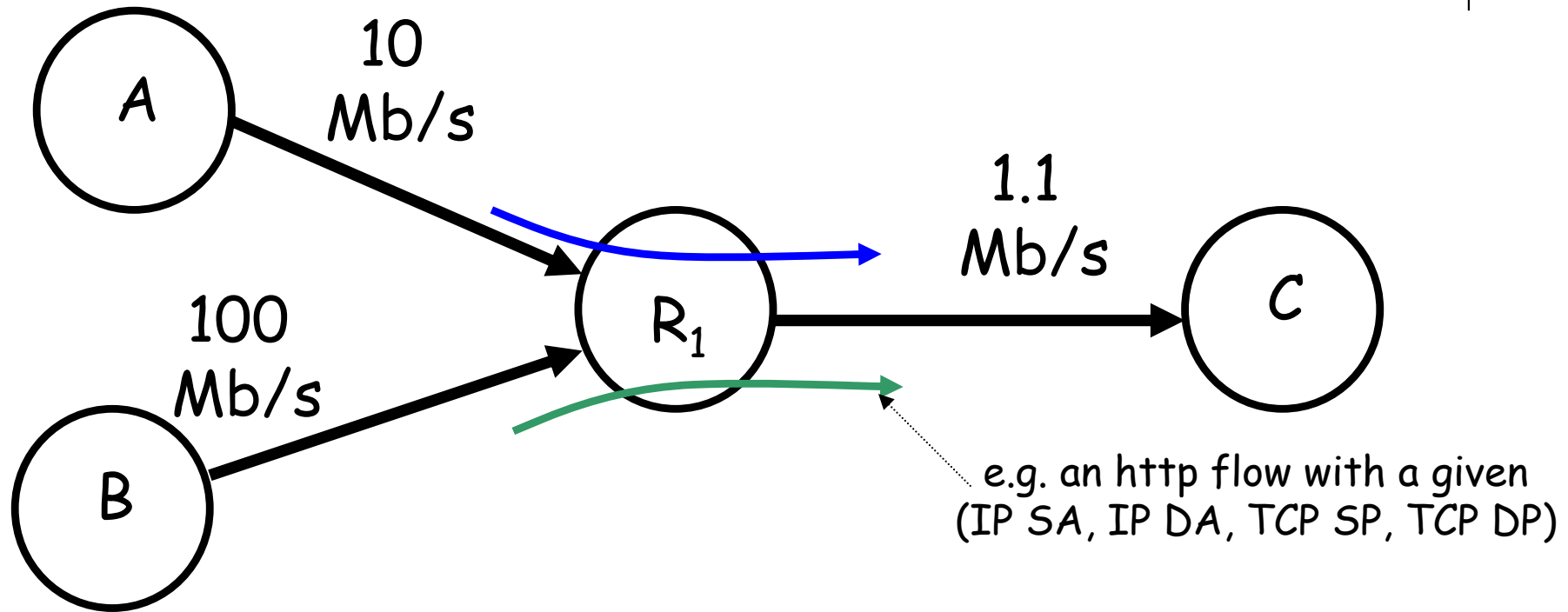


DRR: properties

- Handles variable length packets
- Backlogged source share bandwidth equally
- Preferably, packet size $<$ quantum
- Simple to implement
 - Similar to round robin

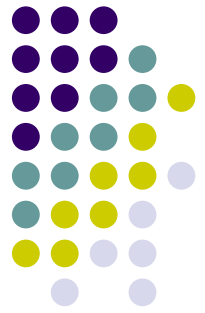


Fairness

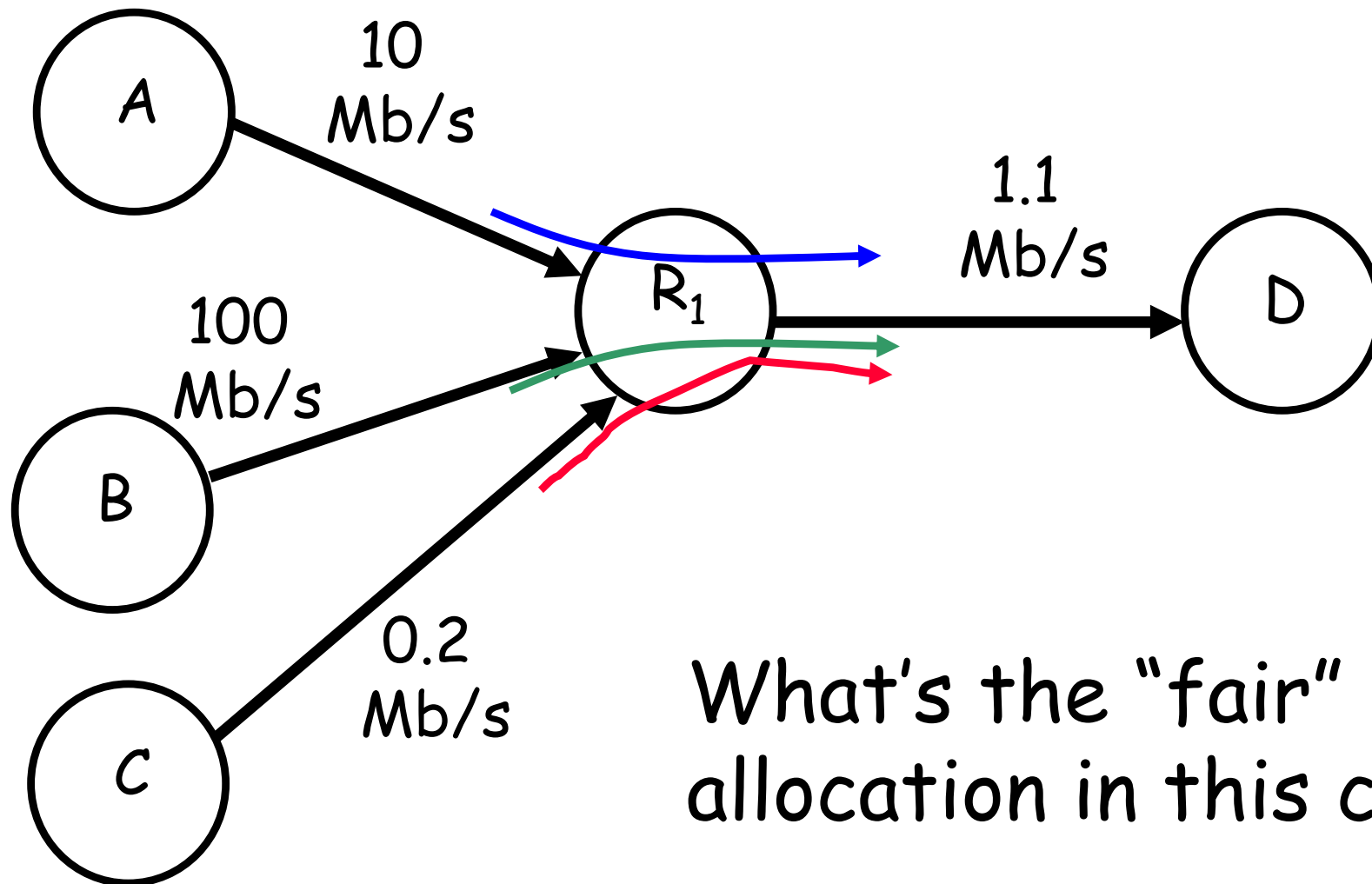


What is the "fair" allocation?

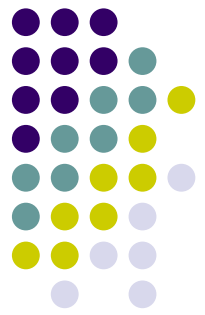
- Equal share (0.55Mb/s, 0.55Mb/s)?
- Proportionate share (0.1Mb/s, 1Mb/s)?



Fairness



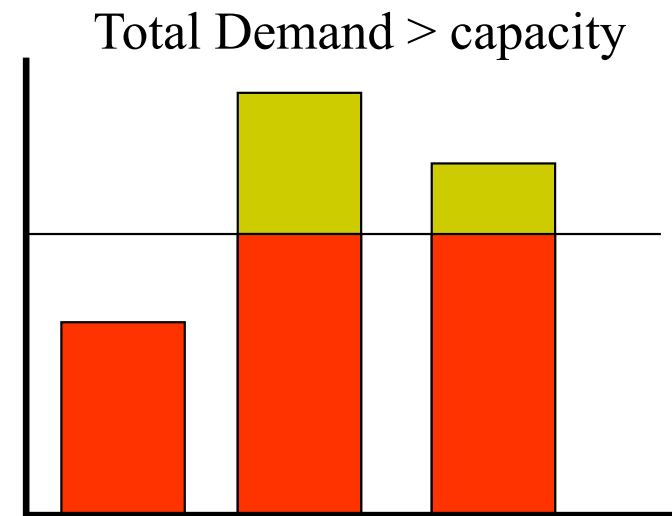
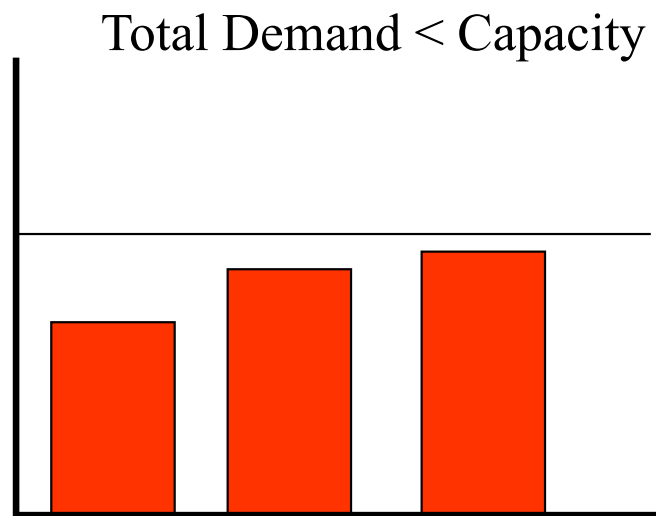
What's the "fair" allocation in this case?

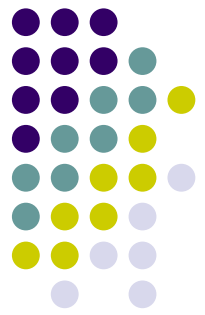


Max-Min Fairness

A common way to allocate flows

- Assume all users have equal *rights* (priority) to a resource:
 - Each connection gets no more than what it wants.
 - Excess is distributed evenly across connections.

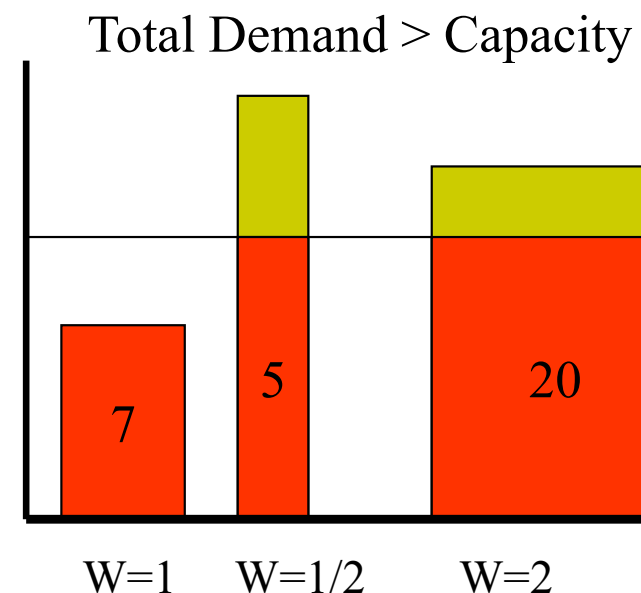
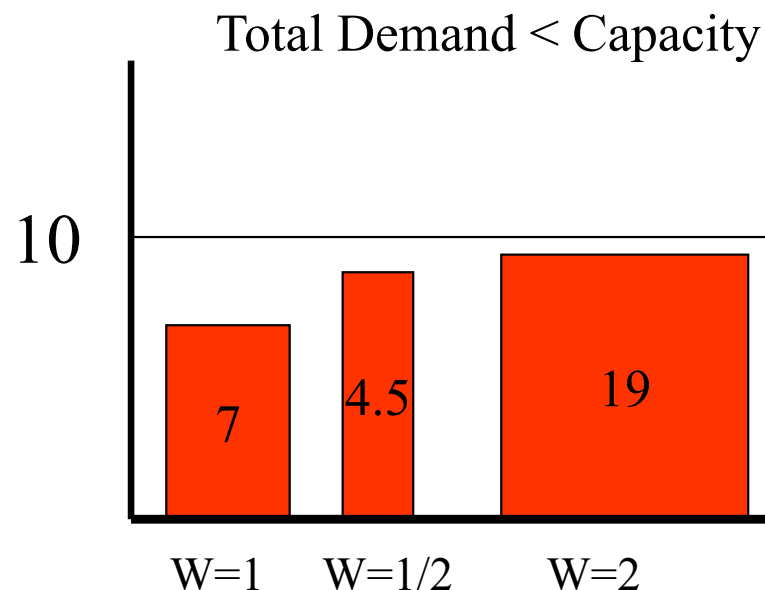


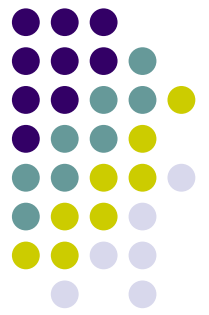


Weighted Max-Min Fairness

Suppose each connection has a *weight*: Some connections are more important than others.

- A connection of weight w should send w bits for each bit a connection of weight 1 sends





Weighted Max-Min Fairness: Algorithm for a single link

Input: link rate C ; and a set of flows, each flow f has

- weight $w(f)$
- demand rate $D(f)$

Requirement: compute allocation $R(f)$ to each flow f .

Let W be the sum of all weights

While $W > 0$ do:

 Let f be the flow minimizing $D(f)/w(f)$

$R(f) := \min(D(f), C \cdot w(f)/W)$ // that's it for f

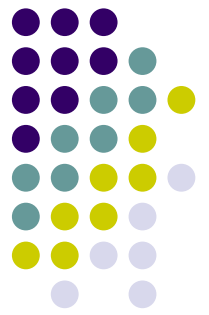
$W := W - w(f)$ // remaining weight

$C := C - R(f)$ // leftover bandwidth

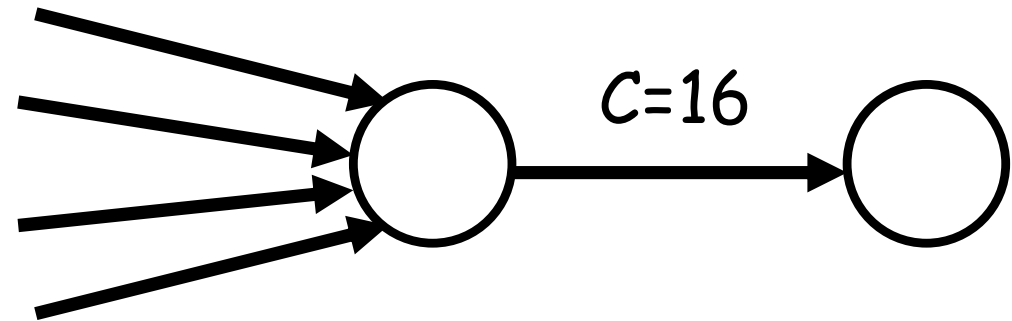
 remove f from the set of flows // f is done

Weighted Max-Min Fairness:

Example run



Flow	demand	weight	D/W
f_1	6	1	6
f_2	8	2	4
f_3	4	4	1
f_4	3	1	3



Initially: $W \leftarrow 8, C \leftarrow 16$

iteration 1: $R(f_3) \leftarrow 4 = \min(4, 16 \cdot 4/8)$

$W \leftarrow 4, C \leftarrow 12$

iteration 2: $R(f_4) \leftarrow 3 = \min(3, 12 \cdot 1/4)$

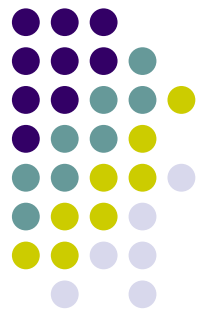
$W \leftarrow 3, C \leftarrow 9$

iteration 3: $R(f_2) \leftarrow 6 = \min(8, 9 \cdot 2/3)$

$W \leftarrow 1, C \leftarrow 3$

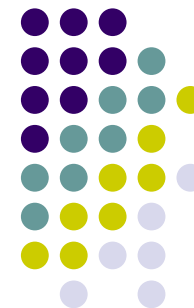
iteration 4: $R(f_1) \leftarrow 3 = \min(6, 3 \cdot 1/1)$

$W \leftarrow 0, C \leftarrow 0$

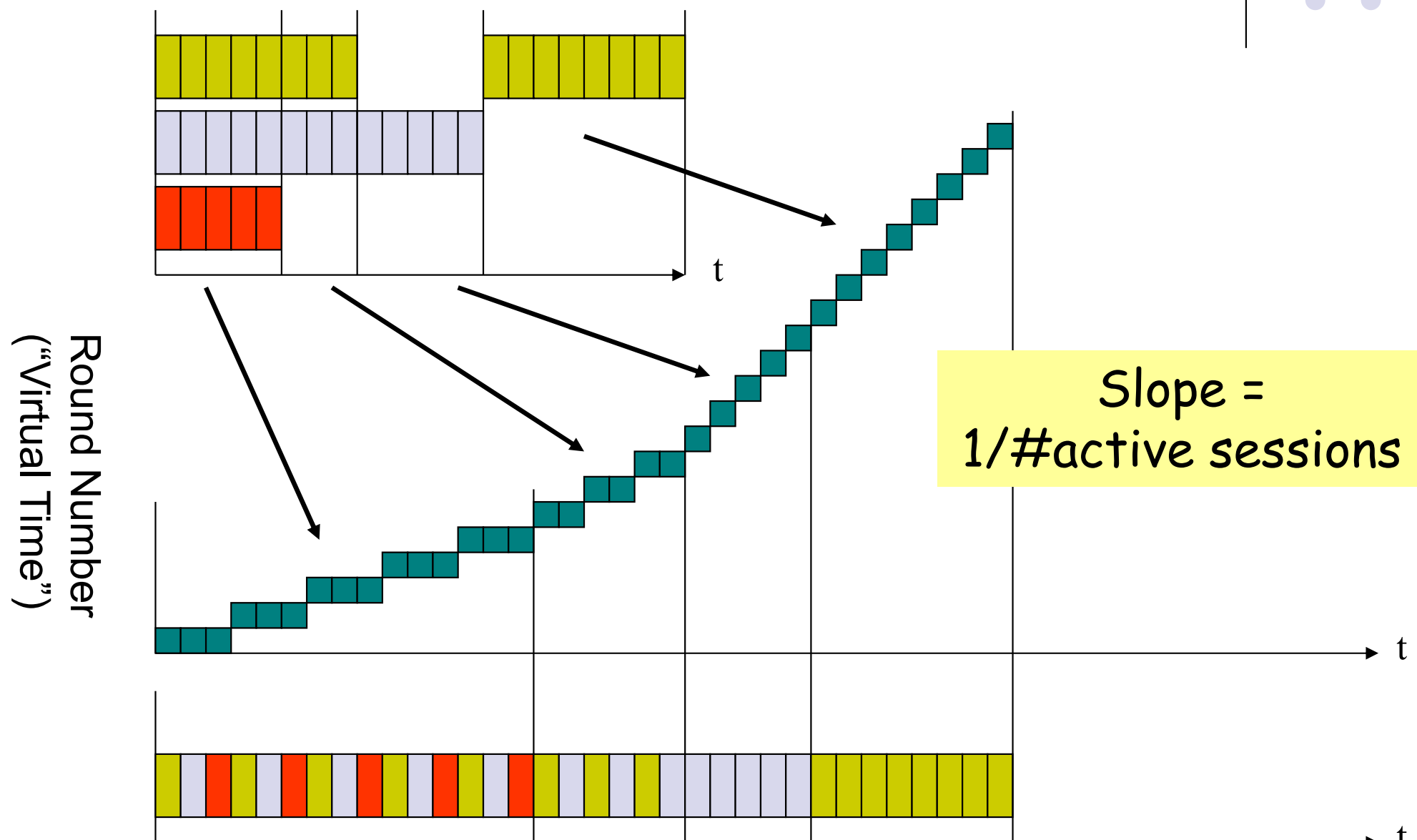


From flows to packets

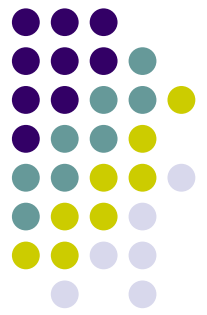
- So far, considered “fluids”, but we need to schedule packets!
- First step:
 - Each packet represents a demand proportional to its size
 - Pretend (for now) that packets can be segmented into bits
 - Do “Bit Round Robin” (BRR)



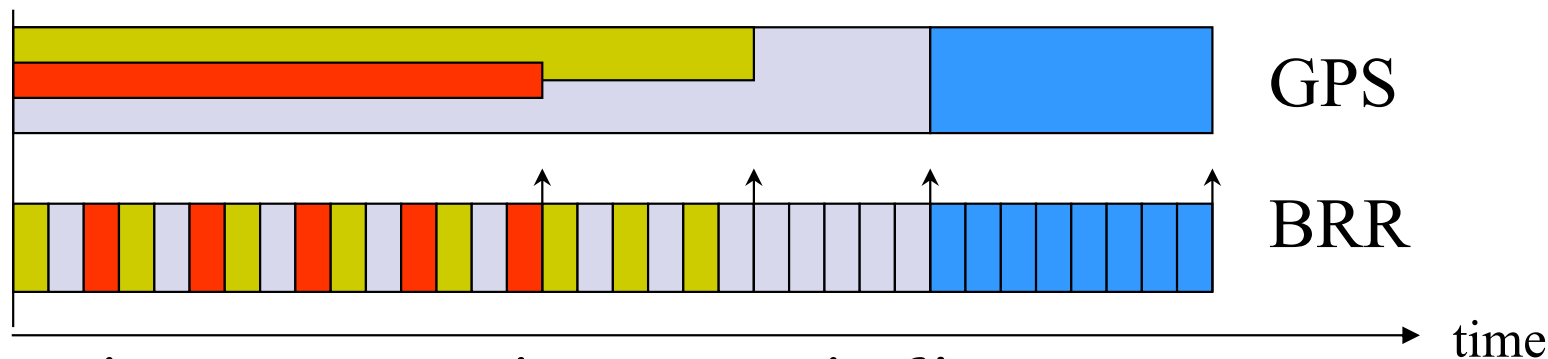
Bit Round Robin Scheduling



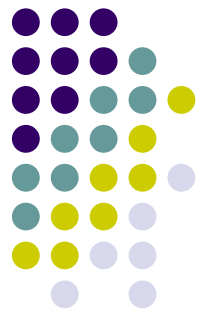
Generalized Processor Sharing



- Processor Sharing - send an infinitesimally small fraction of a bit
 - Like multiple flows sharing the output line simultaneously
 - No flow has to wait for "its turn".
 - Is perfectly fair at all times.



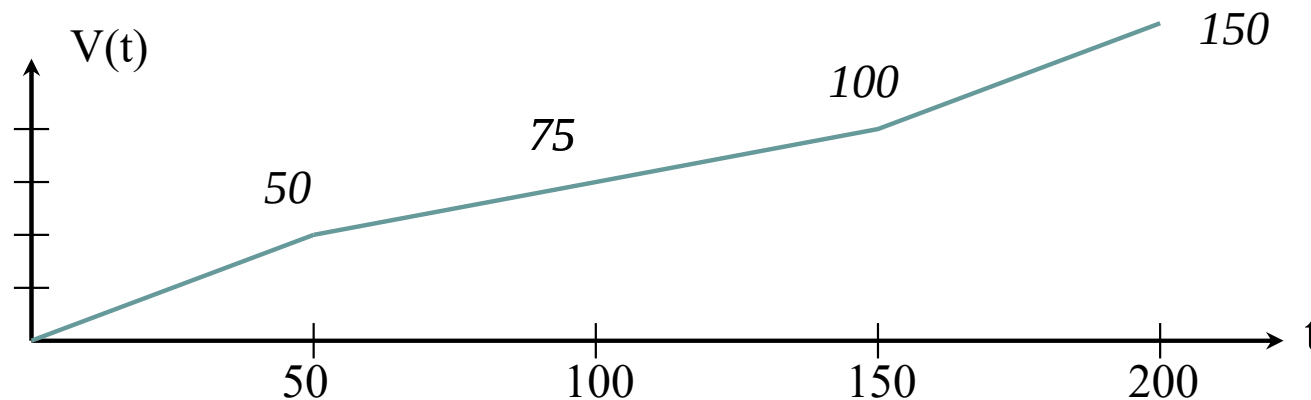
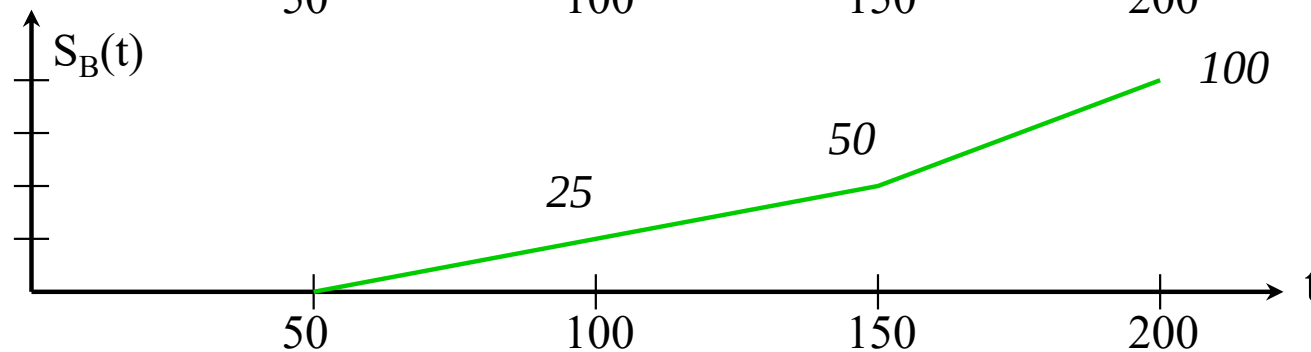
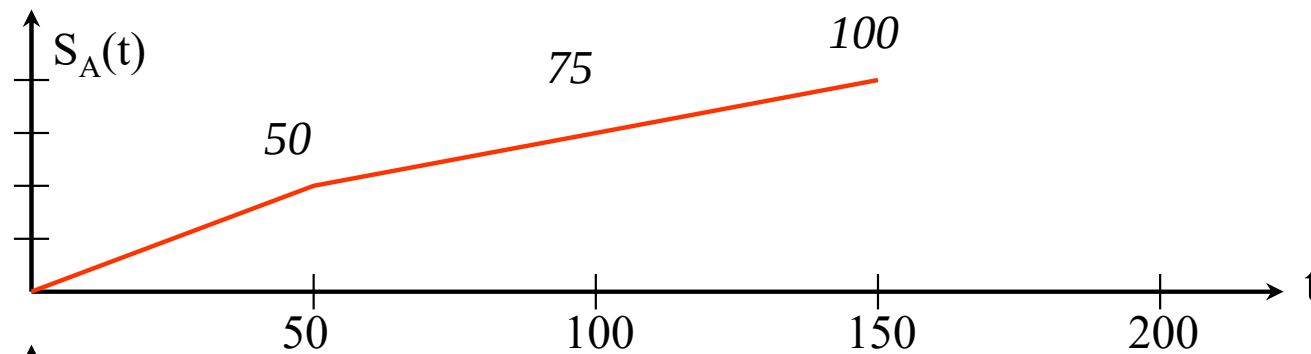
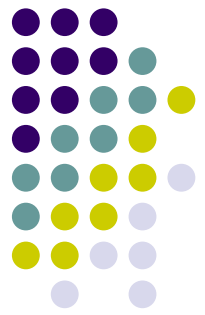
- Can have a *weight* to each flow
 - A flow with weight "2" gets to send two bits per round.



GPS - An Example

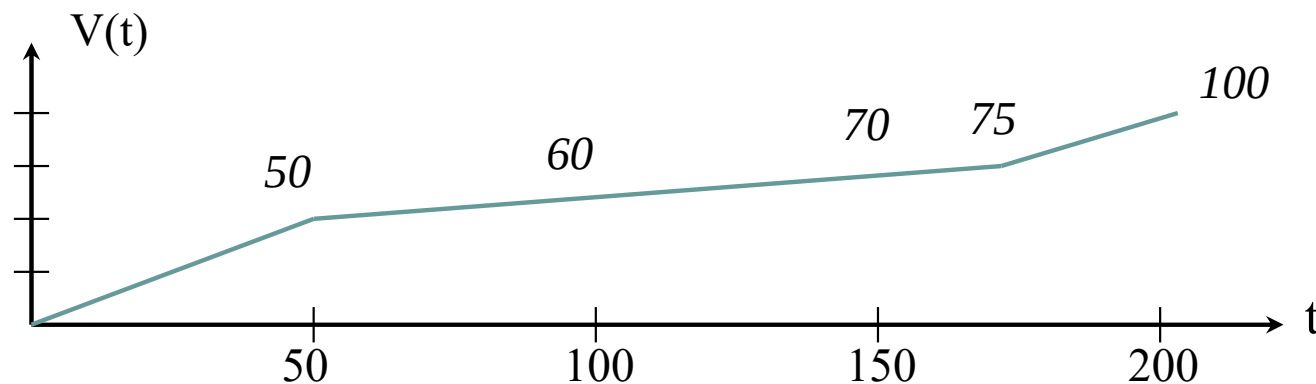
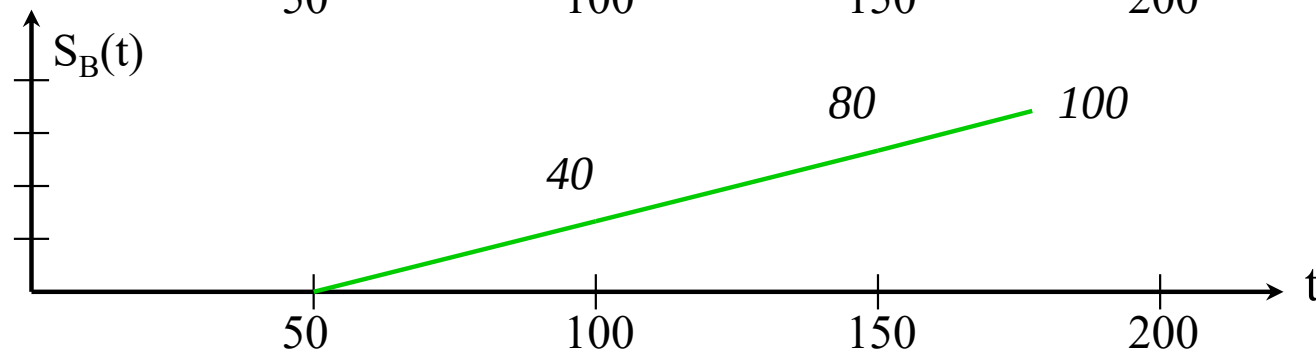
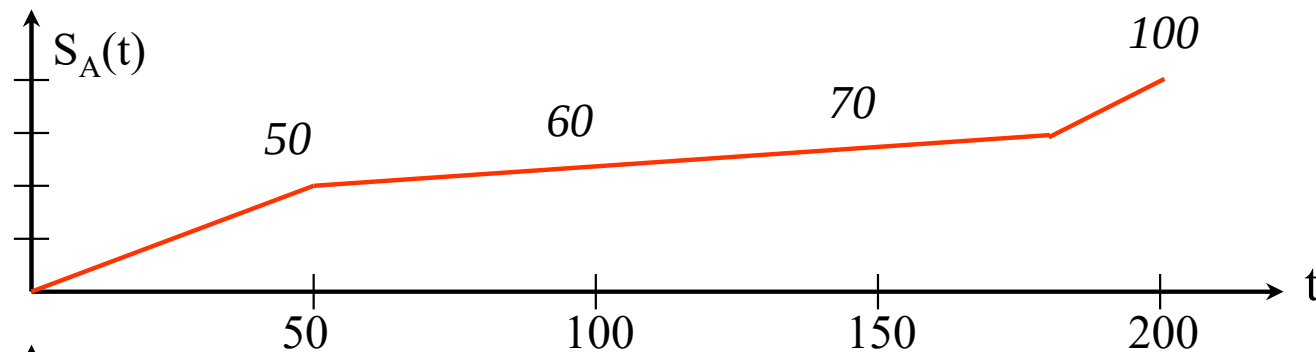
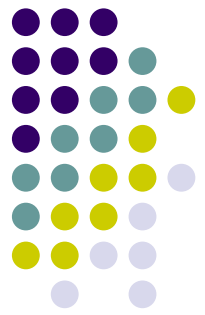
- Packet A arrives at time 0
- Packet B arrives at time 50
- Both packets are 100 bits long.
- Output line rate is 1 b/s.

GPS - An Example



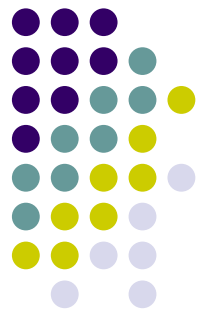
arrival(A)=0,
arrival(B)=50
Length= 100 .
Output speed=1

GPS - A Weighted Example



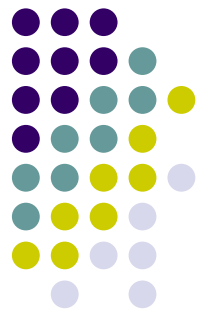
- line speed = 1
- $w(A)=1$
- $w(B)=4$
- $\text{arrival}(A)=0$
- $\text{arrival}(B)=50$
- packet lengths=100

Fairness using GPS



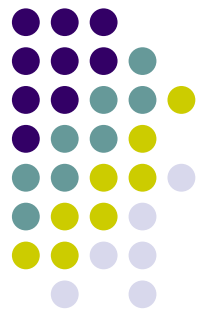
- When there's no contention: connections get what they ask for.
- When there's contention: connections share bandwidth proportionally to their weights.
 - Let $BL(t)$ be the set of **backlogged** connections at time t . Connection $i \in BL(t)$ gets rate

$$R_i(t) = \frac{w_i}{\sum_{j \in BL(t)} w_j}$$



GPS: Benchmark for Fairness

- Define: $S_A(f, t)$ = # bits transmitted for flow f in t time units by policy A .
- **Absolute** fairness bound for policy A :
 - $\max_{f,t} (S_{GPS}(f, t) - S_A(f, t))$
- **Relative** fairness bound for policy A :
 - $\max_{f,f',t} (S_A(f, t) - S_A(f', t))$ for backlogged f, f'

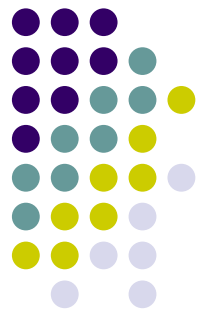


Fairness of Round Robin

For **uniform packet size**, round robin gives:

- Relative bound 1 pkt
- Absolute bound 0 pkt

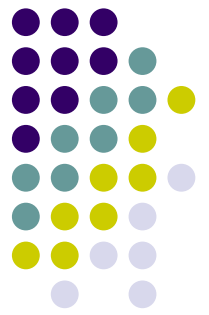
Challenge: handle variable size packets.



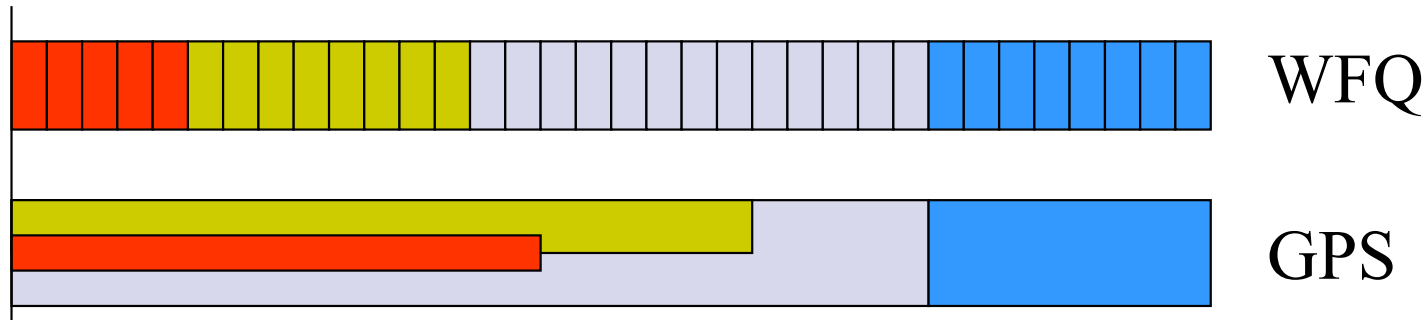
Weighted Fair Queuing (WFQ)

Goal: emulate GPS with packets

- Idea:
 1. Keep track of what GPS would have done
 2. Serve packets in order of their **GPS finish time**
- Cannot always do this unless we know the future...

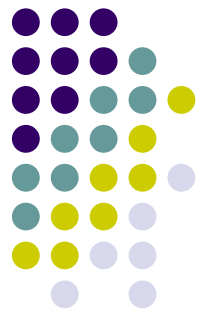


WFQ - Example

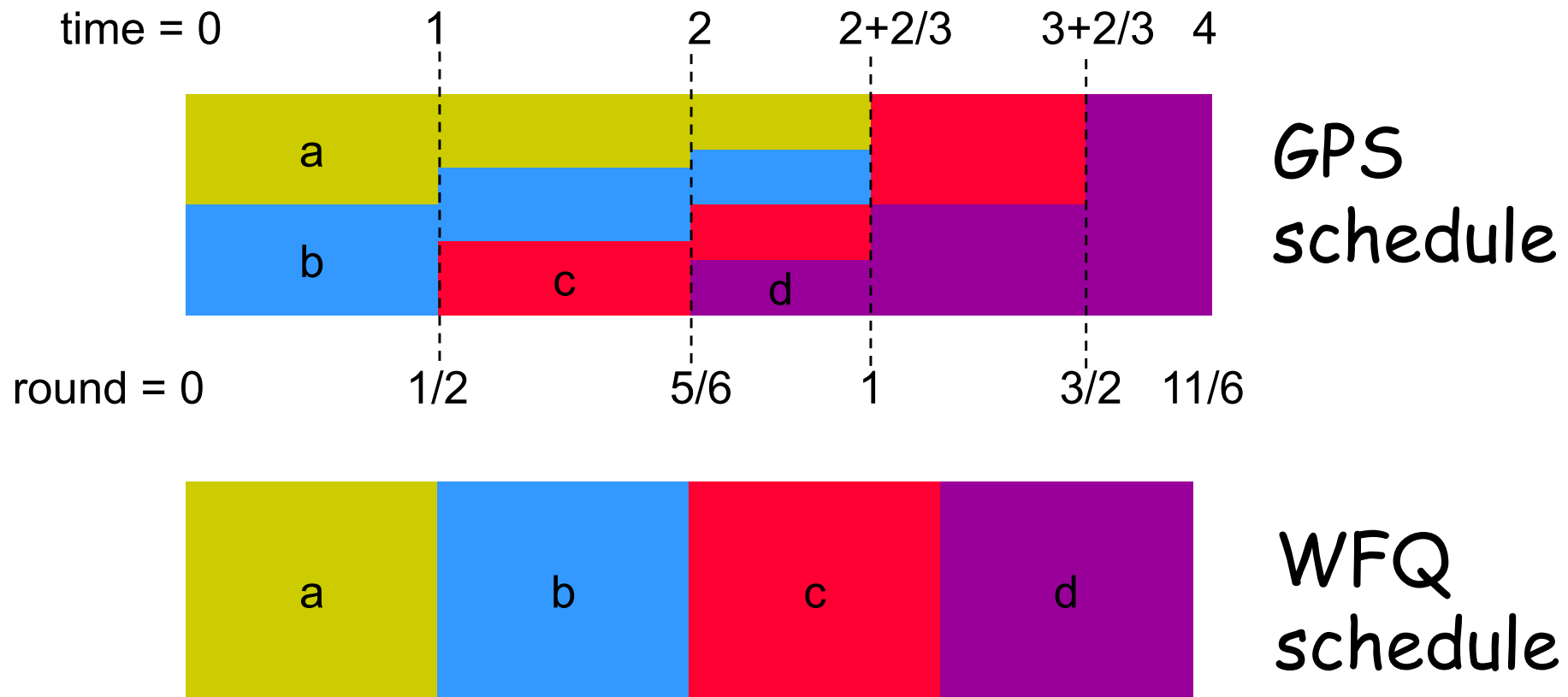


Packets are sent in order of departure in the corresponding GPS system

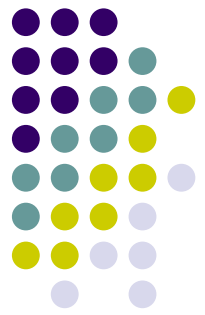
- WFQ (unlike simple round robin) deals with variable packet sizes.



WFQ: Example graphically



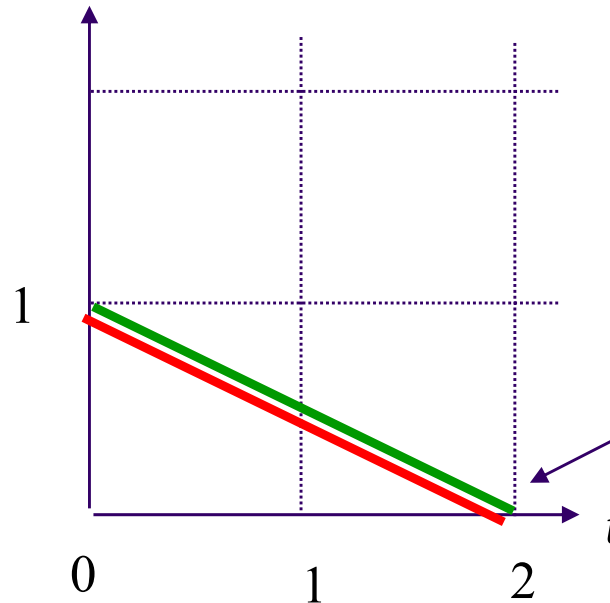
GPS vs WFQ: uniform packets



Queue 1
@ $t=0$



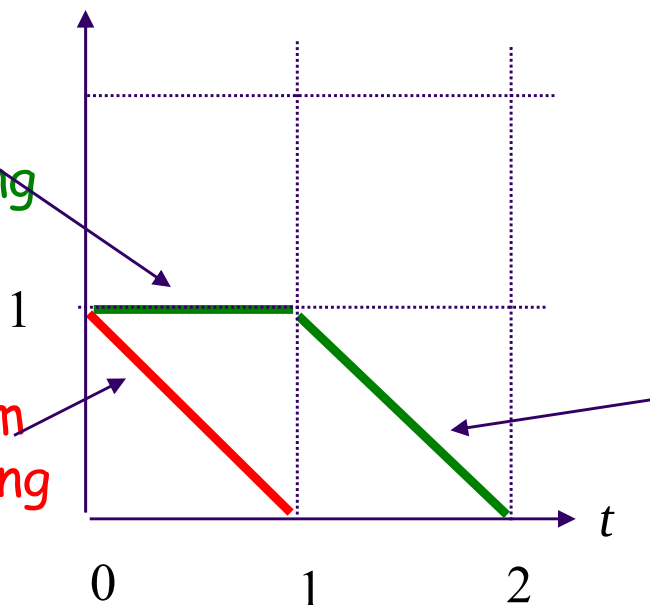
Queue 2
@ $t=0$



GPS: both
packets served
at rate 1/2

Both packets
complete service at
 $t=2$

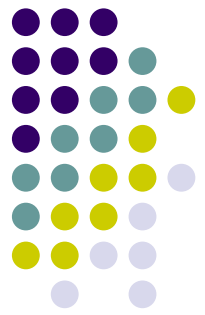
Packet from
queue 2 waiting



Packet-by-packet system (**WFQ**):
queue 1 served first at rate 1;
then queue 2 served at rate 1.

Packet from queue 2
being served

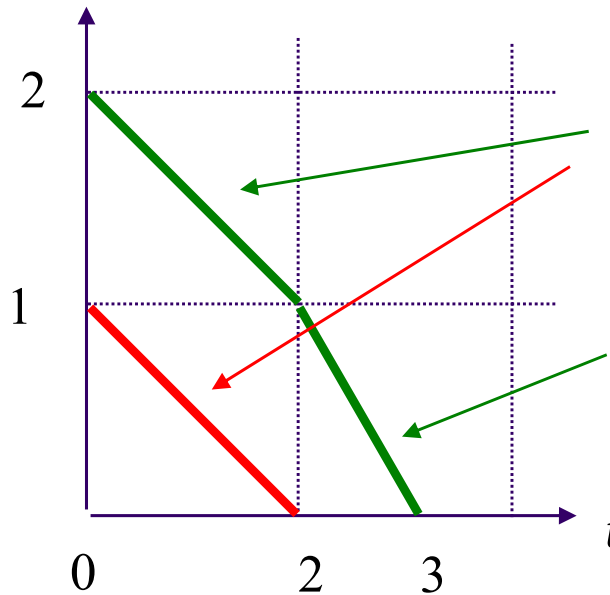
GPS vs WFQ: general packets



Queue 1
@ $t=0$

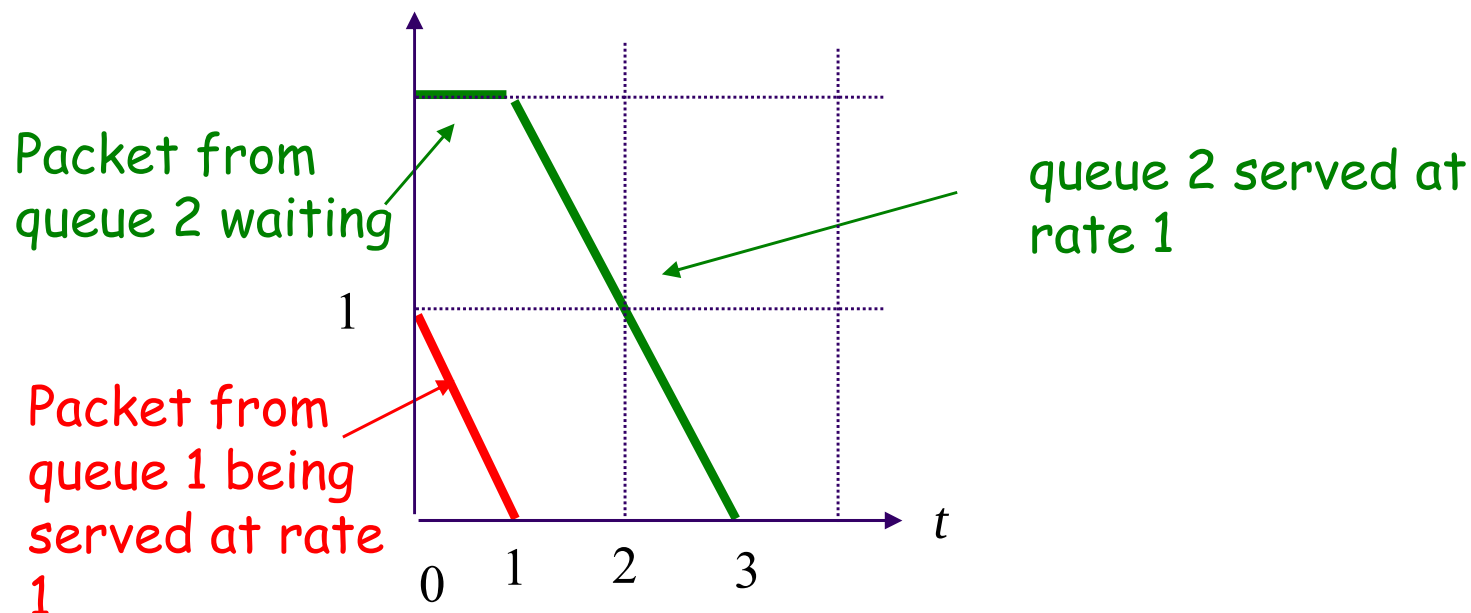


Queue 2
@ $t=0$



GPS: both packets served
at rate $1/2$

Packet from queue
2 served at rate 1

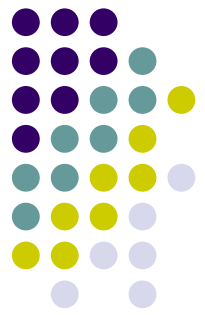


Packet from
queue 2 waiting

Packet from
queue 1 being
served at rate
1

queue 2 served at
rate 1

GPS vs WFQ: weights



Queue 1
@ $t=0$



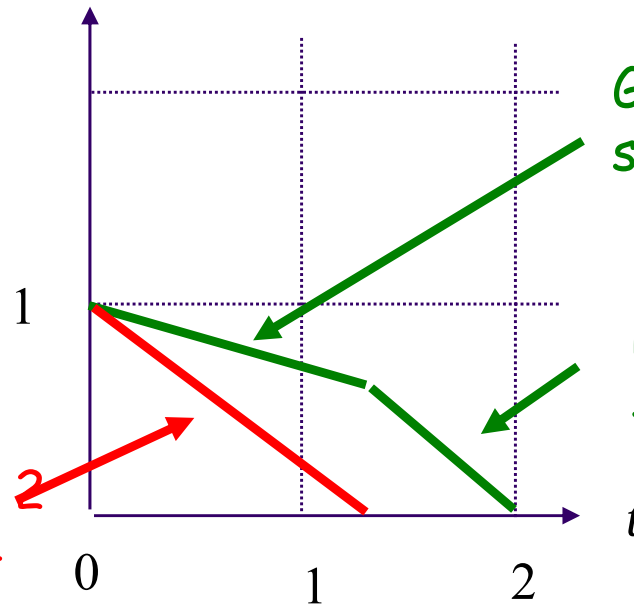
Queue 2
@ $t=0$



Weights:

$Q1=1$
 $Q2=3$

Packet from queue 2
served at rate $3/4$

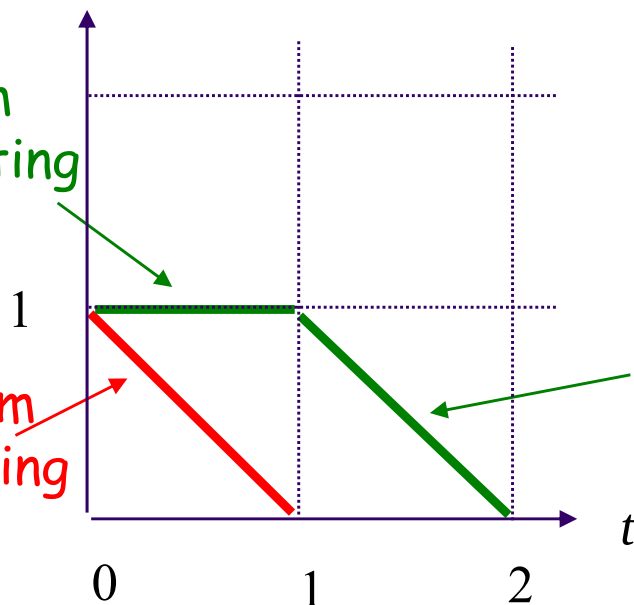


GPS: packet from queue 1
served at rate $1/4$;

Packet from queue 1
served at rate 1

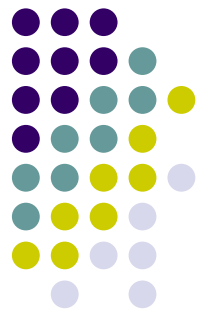
Packet from
queue 1 waiting

Packet from
queue 2 being
served



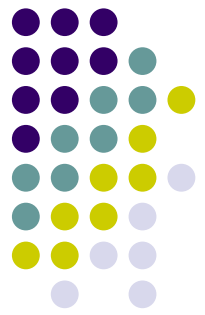
WFQ: queue 2 served first at rate 1;
then queue 1 served at rate 1.

Packet from queue 1
being served

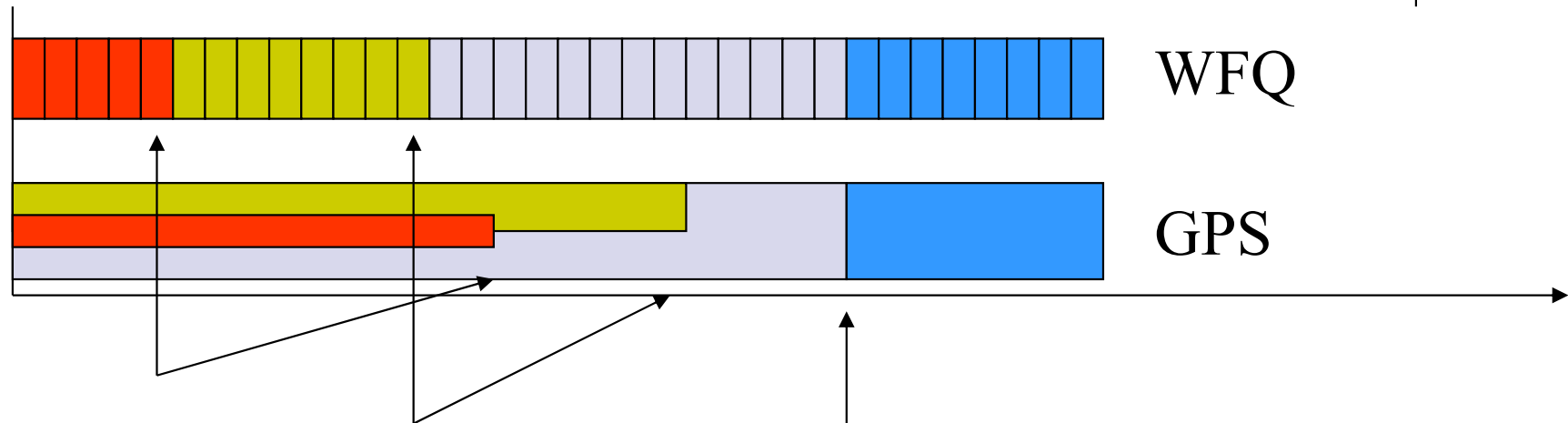


WFQ and GPS: single link

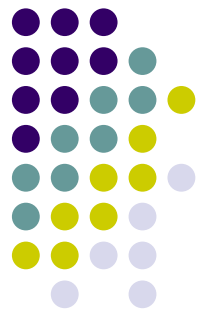
- GPS is never ahead of WFQ by more than one packet (in terms of transmitted bits up to time t).
- Packets in WFQ are not delayed more than one packet length relative to GPS:
 - Bits sent: $S_{\text{GPS}}(f, t) - S_{\text{WFQ}}(f, t) \leq L_{\text{max}}$
 L_{max} : length of longest packet in bits
 - Completion time: $D_{\text{WFQ}}(f, k) - D_{\text{GPS}}(f, k) \leq L_{\text{max}}/C$
 C : link speed in bps



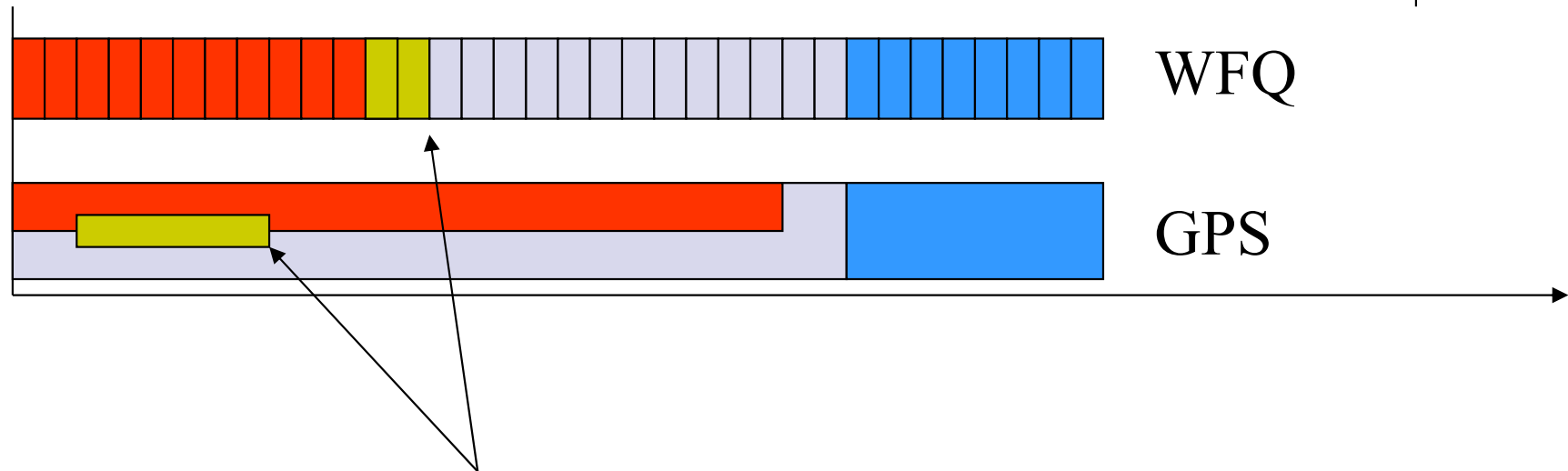
Delay Bound



When WFQ and GPS finish packets at the same order,
WFQ will never lag behind GPS.
(And may sometimes be ahead of GPS).



Delay Bound



When a packet arrives "too late," it may have to wait for the current packet to finish transmission, hence $L_{\max}/C$ delay.

- Single link: Error term does *not* accumulate over time.
- May accumulate over multiple network hops.

GPS and WFQ: Network

Parekh-Gallager theorem

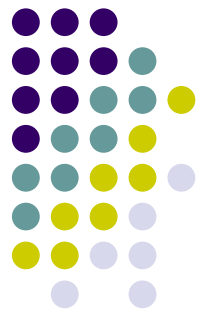


Suppose a given connection is (σ, ρ) constrained, has maximal packet size L , and passes through K WFQ schedulers, such that in the i th scheduler

- there is total rate $r(i)$
- from which the connection gets $g(i)$.

Let g be the minimum over all $g(i)$, and suppose all packets are at most $L_{\max}$ bits long. Then

$$\text{End-to-end delay} \leq \frac{\sigma}{g} + \sum_{i=2}^K \frac{L}{g_i} + \sum_{i=1}^K \frac{L_{\max}}{r(i)}$$



P-G theorem: Interpretation

$$\text{End-to-end delay} \leq \frac{\sigma}{g} + \sum_{i=2}^K \frac{L}{g_i} + \sum_{i=1}^K \frac{L_{\max}}{r(i)}$$

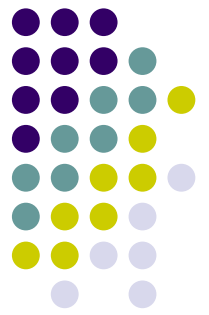
Delay of last packet of a burst. Only in first node

GPS term

store&forward penalty: not in first node

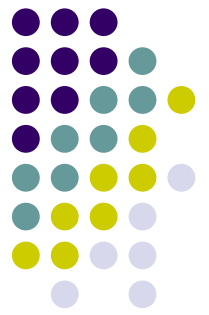
WFQ lag behind GPS: each node

GPS to WFQ correction



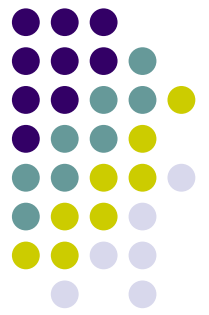
Significance

- WFQ can provide end-to-end delay bounds
- So WFQ provides **both** fairness and performance guarantees
- Bound holds **regardless** of cross traffic behavior
- Can be generalized for networks where schedulers are variants of WFQ, and the link service rate changes over time



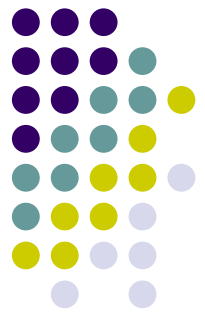
Fine Points

- To get a delay bound, need to pick g
 - the lower the delay bound, the larger g needs to be
 - large g means exclusion of more competitors from link
- Sources must be leaky-bucket regulated
 - but choosing leaky-bucket parameters is problematic
- WFQ couples delay and bandwidth allocations
 - low delay requires allocating more bandwidth
 - wastes bandwidth for low-bandwidth low-delay sources

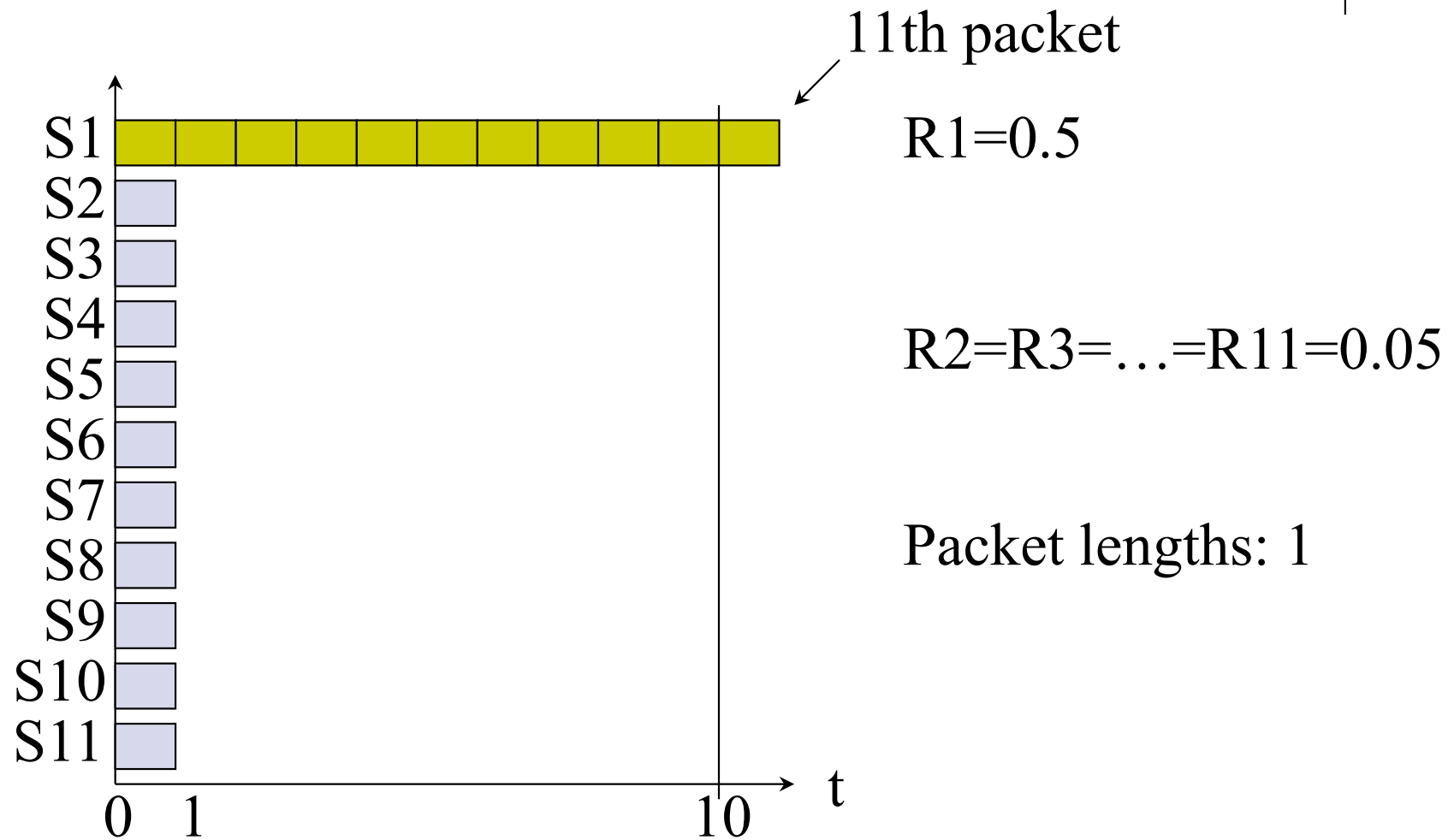


Worst Case WFQ (WF²Q)

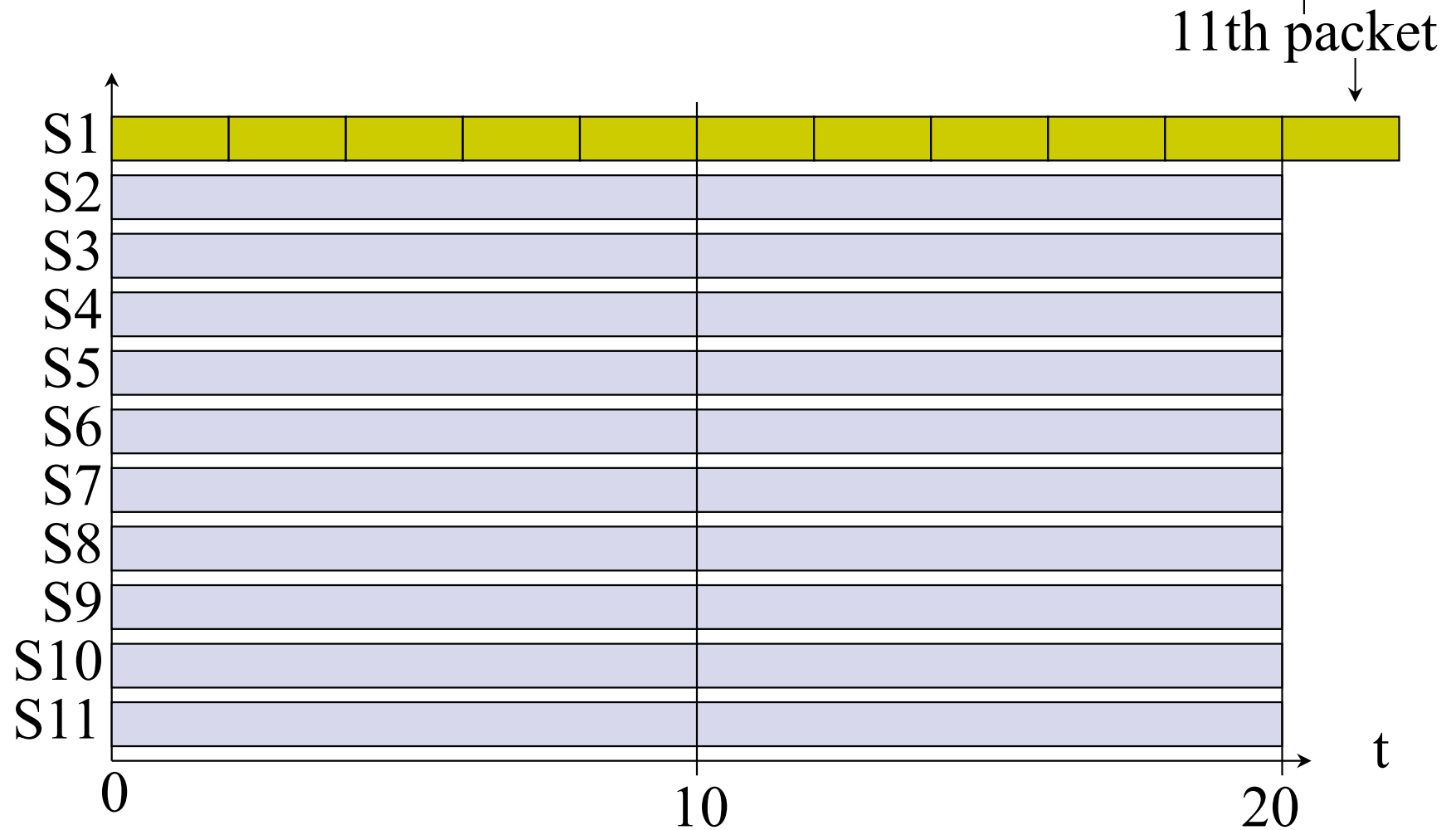
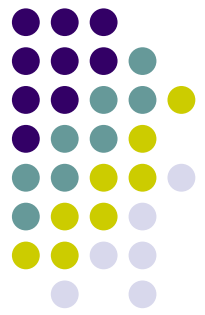
- We've just said that WFQ approximates GPS to within a difference of one packet.
- That was not quite true...
 - The bound is one way: WFQ is never **lagging** by too much, $S_{WFQ}(f, k) - S_{GPS}(f, k) \leq L_{\max}$
- But WFQ might be well *ahead* of GPS!



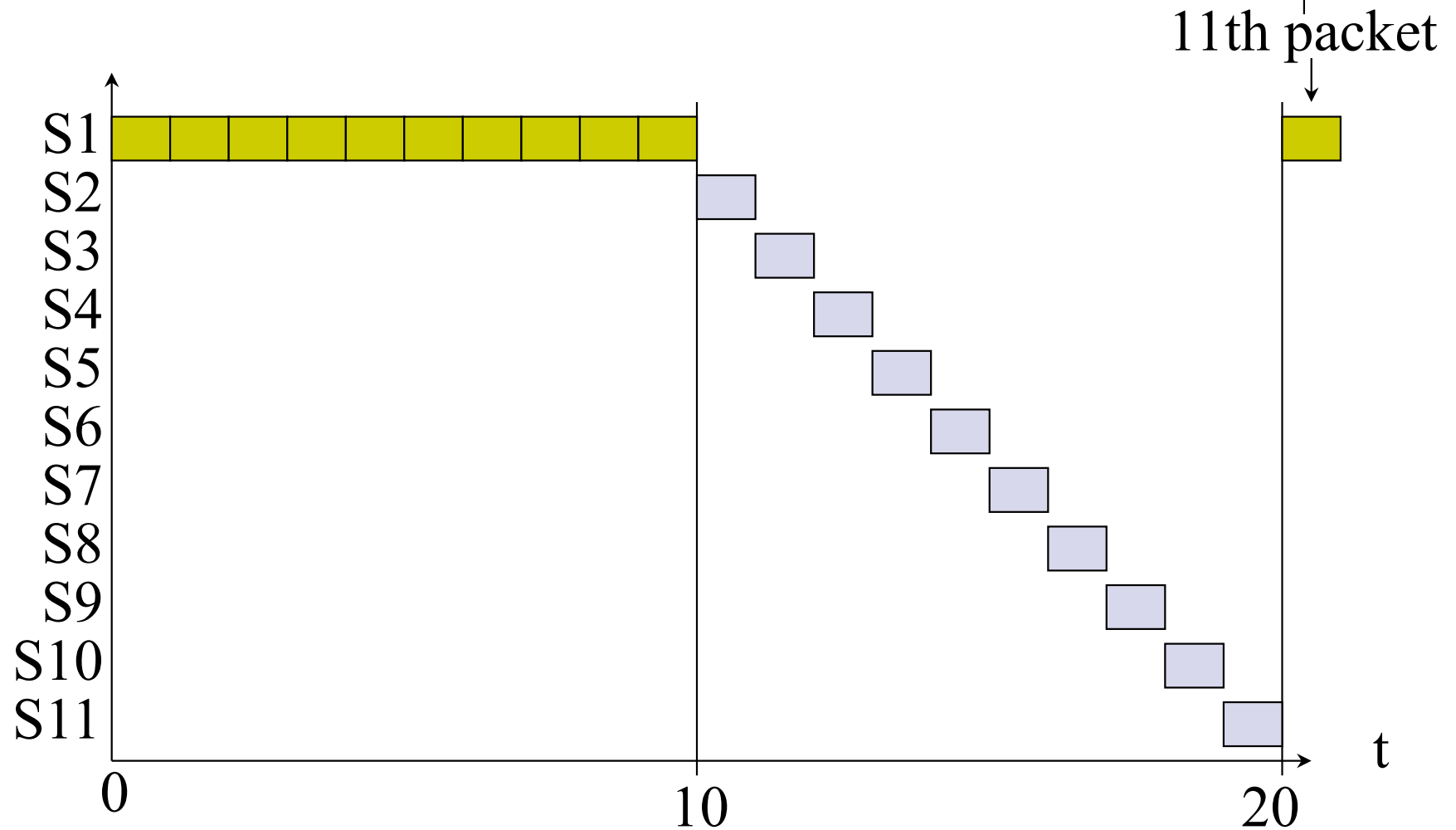
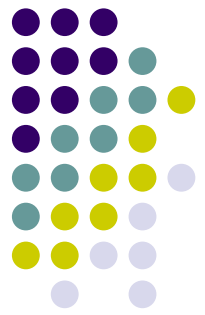
WF²Q - Example



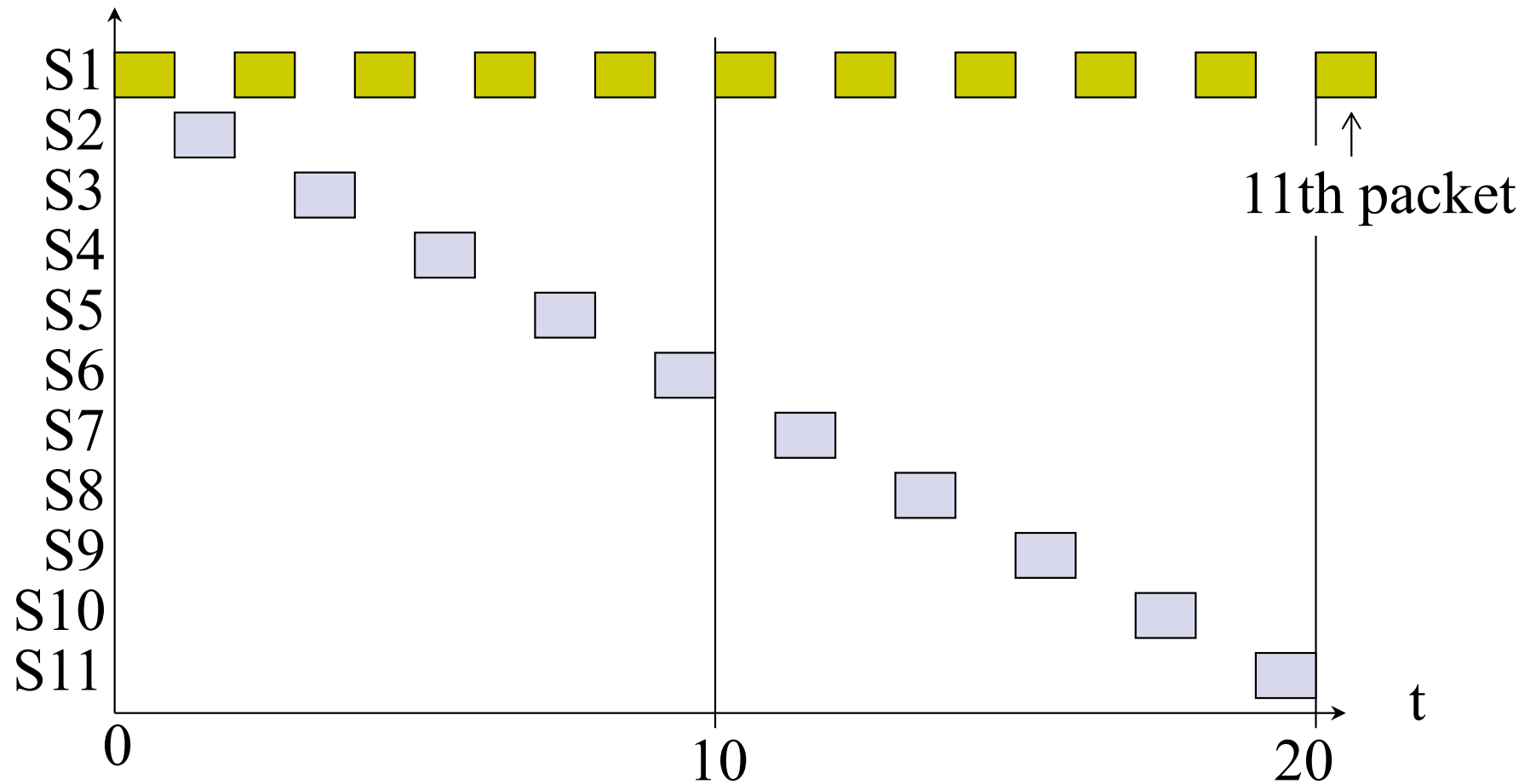
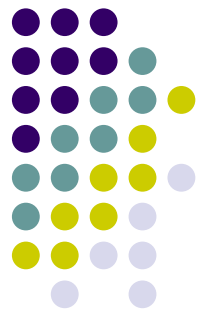
GPS Service Order

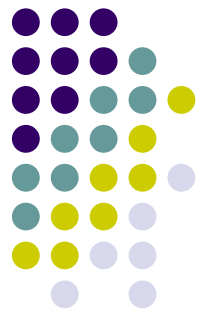


WFQ Service Order



WF²Q Service Order

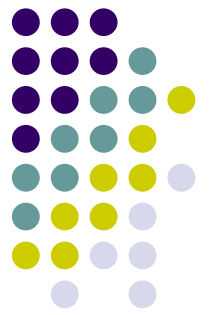




WF²Q Idea

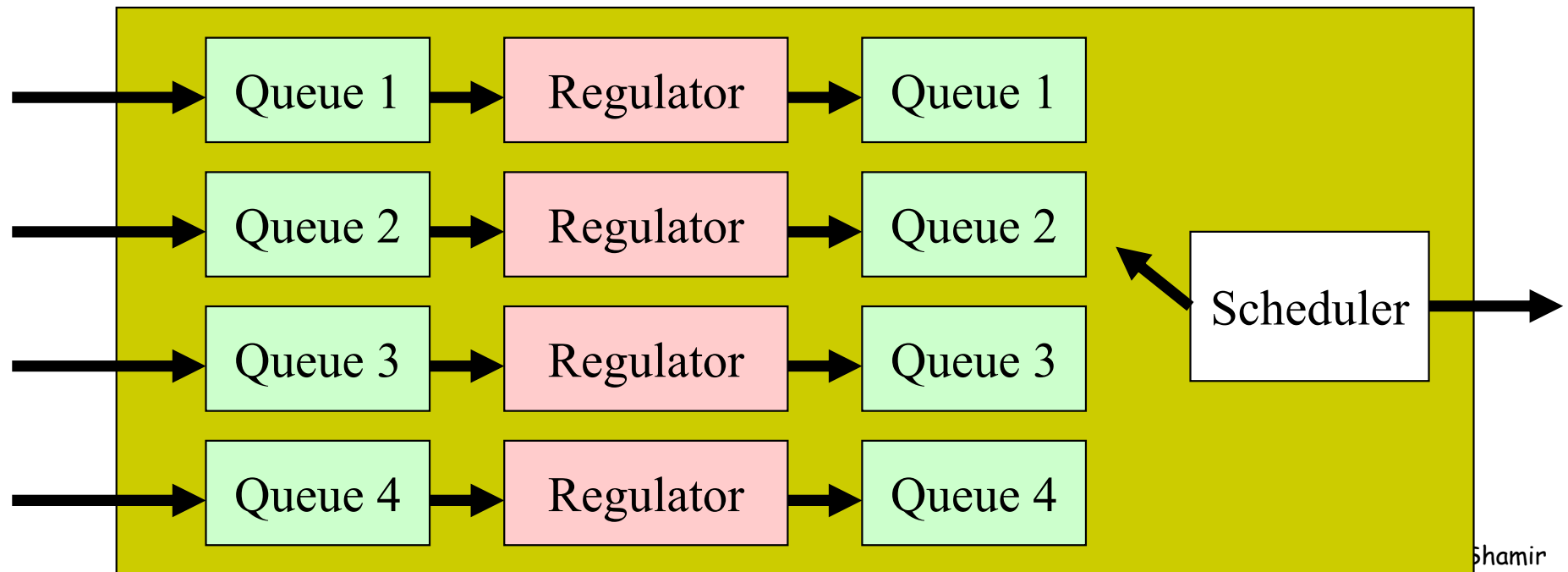
- WFQ: scheduler selects next packet with minimal finish number from *all available packets*.
 - Worst-case fair weighted fair queuing (WF²Q): scheduler considers *only packets that have started in the emulated GPS system*.
- ⇒ Get closer emulation of GPS: add lower bound on the time difference with GPS

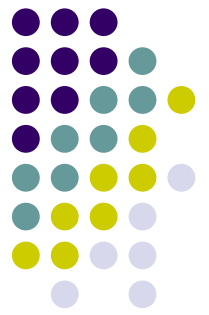
Implementation: Rate Controlled Scheduling



- Regulator holds packets until they are *eligible* for transmission: use **GPS start** time.
- Scheduler decides which eligible packet should be transmitted next: use **GPS finish** time.

Switch Output Module





WF²Q Properties

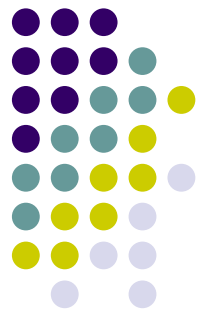
- Retain upper bound of WFQ:

$$S_{i,\text{GPS}}(0, \tau) - S_{i,\text{WF}^2\text{Q}}(0, \tau) \leq L_{i,\text{max}}$$

- Add lower bound:

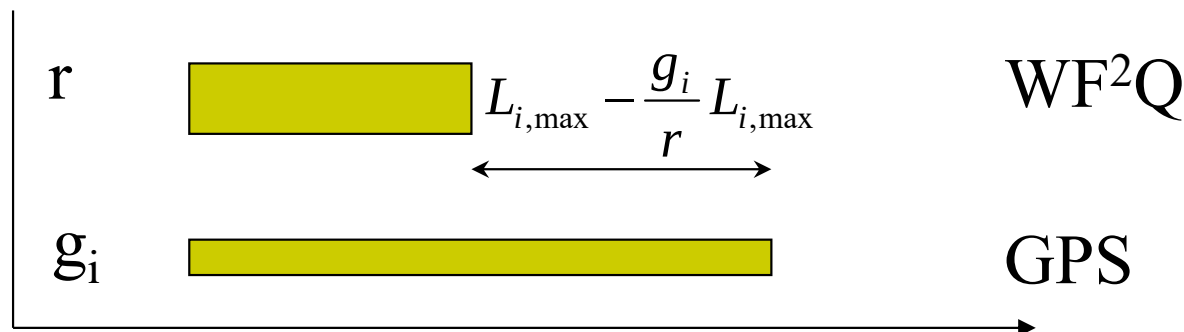
$$S_{i,\text{WF}^2\text{Q}}(0, \tau) - S_{i,\text{GPS}}(0, \tau) \leq \left(1 - \frac{g_i}{r}\right) L_{i,\text{max}}$$

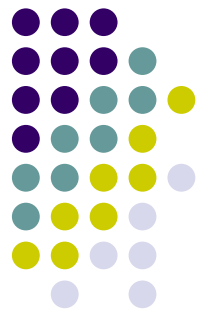
- Best possible in general!



WF²Q Properties (cont.)

- Discipline is *work-conserving*:
 - Rate controller + GPS scheduler is the same as GPS alone.
 - Replace GPS with WFQ (both work conserving) to get WF²Q.
- Lower bound
 - WF²Q starts sending a packet no earlier than GPS.
 - GPS usually takes more time to transmit the packet (depending on allocated and available bandwidth).





New Concepts

- Priority scheduling
- Weighted round robin (WRR)
- Deficit round robin (DRR)
- Weighted max-min fairness
- Generalized processor sharing (GPS)
- Worst-case fair WFQ (W^2FQ)